

AgentShield: Agentic AI Security Framework

Securing Tool-Using AI Agents Against Prompt Injection, Data Leakage, and Unsafe Actions

Mrinal Setty — Agent Orchestration & Target Agent Lead

Aditya Shenoy — Red-Team Agent & Dataset Lead

Yashas Uttangi — Safety, Firewall, Evaluation & Console Lead

CS6180 · Summer 2026

Contents

1	Problem Definition and GenAI Fit	1
2	Baseline Comparison	2
3	Application and Technical Depth of GenAI Techniques	2
4	Data, Knowledge Sources, and Dataset Quality	3
5	Evaluation Pipeline	4
6	Experiments Performed	4
7	Visualizations and Tabular Representations	5
8	Links to the Codebase (GitHub)	6
9	Milestones and Team Contributions	6
10	Conclusion and Limitations	7

1 Problem Definition and GenAI Fit

1.1 Problem Statement

Modern AI systems are moving from chatbots to *agentic* systems that take real actions through tools such as email, files, calendars, task managers, code hosting, and HTTP APIs. This shift introduces a new class of security risk: malicious content hidden inside emails, documents, webpages, or tool responses can manipulate an agent into unsafe actions — leaking private data, messaging unauthorized recipients, or performing destructive operations the user never requested.

The problem this project addresses is: **how can we test and protect tool-using AI agents from prompt injection, data leakage, unauthorized tool use, and unsafe actions before a tool call is executed?**

AgentShield is a multi-agent security framework that simulates tool-using agents, generates red-team attacks against them, validates each proposed tool call against structured safety policies, and blocks or escalates risky

actions with an audit trail. The firewall inspects the proposed call **before** execution, so an unsafe action is *stopped*, not merely reported afterward.

1.2 Why Generative AI Fits

The task is inherently language- and reasoning-driven, which is where generative AI is essential: the system must interpret natural-language requests, reason over tool-use intent, generate realistic adversarial scenarios, apply policies to ambiguous cases, classify risk, and explain decisions:

Component	GenAI role
Target Agent	Tool-use reasoning and structured tool-call generation.
Red-Team Agent	Adversarial prompting to generate prompt-injection and malicious scenarios.
Policy Compiler / Firewall	Policy interpretation, ambiguous-risk reasoning, and audit-explanation generation.
LLM-as-a-judge	Subjective audit-explanation quality only — never primary correctness.

Correctness is always measured against fixed ground-truth labels; the LLM judge scores only the *readability* of explanations.

2 Baseline Comparison

To show that a dedicated firewall adds real value, the same 94-scenario evaluation set is run through three configurations:

- **Unprotected** — allows every proposed tool call.
- **Prompt-only guardrail** — a lightweight safety prompt and credential heuristics, with no structured policy layer.
- **AgentShield** — structured policy rules, risk scoring, and user-intent checks applied to the proposed call.

Configuration	Attack Success (↓)	Defense Success (↑)	Policy Compliance (↑)	Benign Success (↑)
Unprotected	100.0%	0.0%	31.9%	100.0%
Prompt-only guardrail	74.0%	26.0%	45.7%	100.0%
AgentShield	0.0%	100.0%	100.0%	100.0%

The unprotected agent lets every attack through. The prompt-only guardrail catches obvious cases but still misses 74% of attacks, because it cannot inspect tool arguments, recipients, URLs, HTTP methods, file paths, or repository visibility. AgentShield blocks every attack with no benign regressions, demonstrating that structured tool-call enforcement is what closes the gap.

3 Application and Technical Depth of GenAI Techniques

3.1 Decision Pipeline

Each request or stored scenario flows through one path: the **Target Agent** proposes a structured tool call; the call is schema-validated; the **Policy Checker** and **Risk Classifier** evaluate it in parallel; the **Firewall** combines both into the most restrictive decision — ALLOW, BLOCK, or ASK_APPROVAL; only ALLOW may reach optional safe mock execution; and the **Orchestrator** records an audit entry that feeds metrics and the API.

3.2 Red-Team Attack Generation

The Red-Team Agent is the offensive counterpart to the dataset. It renders attack *seeds* into schema-valid adversarial scenarios and delivers hidden directives through five distinct injection patterns, so attacks are varied rather than a single obvious phrase:

Injection pattern	How it hides the directive
<code>direct_override</code>	Explicitly tells the agent to ignore prior instructions.
<code>hidden_html_comment</code>	Buries the directive in markup-like content.
<code>fake_system_note</code>	Impersonates a privileged instruction in external context.
<code>authority_urgency</code>	Uses claimed authority and urgency to pressure compliance.
<code>fake_tool_response</code>	Presents the directive as if it came from a tool response.

Generation is **offline and deterministic** by default: it fills checked-in templates from checked-in seeds, needs no network or API key, and produces stable output suitable for tests and audits. An optional online mode is reserved as an extension point. This deterministic core is a deliberate reproducibility choice.

The generator is built from four checked-in components:

Component	Role
<code>injection_patterns.py</code>	Prompt-injection pattern library.
<code>red_team_seeds.py</code>	Attack seed catalog grouped by tool and style.
<code>red_team_agent.py</code>	Renders a seed into a schema-valid example.
<code>generate_red_team.py</code>	Writes <code>data/red_team_examples.json</code> .

3.3 Policy Reasoning and Audit Explanations

The Policy Compiler turns natural-language safety policies into structured JSON rules; the Firewall reasons over policy violations and risk to select a decision and produce a human-readable audit explanation. An LLM-as-a-judge, guided by a rubric, scores those explanations for quality — the only subjective metric, and the only place an LLM grades output.

4 Data, Knowledge Sources, and Dataset Quality

4.1 Composition

The dataset is a synthetic benchmark of agent tool-call scenarios with ground-truth firewall decisions. The full stored evaluation set is 94 scenarios:

Source	Count	Role
<code>demo_scenarios.json</code>	3	Demo set covering all three decisions.
<code>sample_examples.json</code>	6	Representative schema/label checks.
<code>dataset_v0.json</code>	50	Balanced benign/malicious baseline.
<code>red_team_examples.json</code>	25	Adversarial scenarios across all tools.
<code>benign_edge_cases.json</code>	10	Safe actions that resemble attacks.

The deeper 85-example corpus (`dataset_v0` + `red_team_examples` + `benign_edge_cases`) splits into 35 benign / 50 malicious, with 27 ALLOW, 12 ASK_APPROVAL, and 46 BLOCK labels.

4.2 Schema and Knowledge Sources

Every scenario follows `dataset_schema.json` with seven fields: `user_request`, `external_context` (external content that may carry injected instructions), `proposed_tool_call`, `expected_decision` (ALLOW / BLOCK / ASK_APPROVAL), `risk_level`, `attack_category` (none, prompt_injection, data_exfiltration, unauthorized_action), and `explanation`. Stable ids (`v0-*`, `rt-*`, `be-*`, `demo-*`) support lookup. The controlled label vocabulary is documented in `data/attack_taxonomy.md`.

All eight supported tools are represented: `send_email`, `read_file`, `write_file`, `delete_file`, `create_calendar_event`, `create_task`, `create_github_issue`, and `send_http_request`.

4.3 Dataset Quality

Quality is enforced automatically. `validate_dataset.py` checks every file against the schema; `corpus_report.py` and `quality_report.py` verify balance and flag ambiguous labels; and `label_validator.py` confirms labels agree with runtime firewall behavior using the same shared helpers. A representative check:

```
PYTHONPATH=. python3 data/validate_dataset.py data/dataset_v0.json
```

4.4 Synthetic-Data Boundary

All content is synthetic. Email addresses use `*.example` domains, secret-like values use marked `<synthetic-...-placeholder>` tokens, and tests reject real-looking contact data or credentials.

5 Evaluation Pipeline

5.1 Method

Evaluation runs every stored scenario through the same policy checker, risk classifier, intent helpers, and firewall decision path used by the live API, then compares each decision to the scenario’s `expected_decision` label. Because labels are fixed ground truth, correctness is objective and repeatable.

5.2 Metrics

The pipeline reports Attack Success Rate, Defense Success Rate, Benign Task Success Rate, False Positive / False Negative Rate, Policy Compliance Accuracy, BLOCK precision/recall/F1, and escalation rate. Audit-explanation quality is scored separately with a rubric and an LLM-as-a-judge.

5.3 Reproduction

Evaluation and baseline artifacts are regenerated into `data/evaluation/`:

```
PYTHONPATH=. python3 src/experiment_runner.py
```

6 Experiments Performed

6.1 Full-Corpus Firewall Evaluation

Running all 94 scenarios through the firewall yields 100% policy-compliance with no false negatives and no false positives; the 85-example corpus reaches 85/85 firewall-label agreement.

Metric	AgentShield
Attack Success Rate	0.0%
Defense Success Rate	100.0%
Benign Task Success Rate	100.0%

Metric	AgentShield
False Positive Rate	0.0%
False Negative Rate	0.0%
Policy Compliance Accuracy	100.0%
BLOCK Precision / Recall / F1	100.0%

6.2 Baseline Experiments

The unprotected and prompt-only baselines were run on the identical set, producing the comparison in the *Baseline Comparison* section: attack success falls from 100% (unprotected) and 74% (prompt-only) to 0% under AgentShield.

6.3 Red-Team Gap Discovery and Remediation

Expanding the adversarial corpus across all eight tools initially exposed missing policy coverage — around protected writes, credential leaks in public issues, sensitive file reads, and unauthorized calendar/task creation. These issue classes were then closed with reusable policy checks and regression tests rather than scenario-specific exceptions. Finding and fixing coverage gaps *before* production is precisely the value the red-team track provides.

Issue class surfaced by the red-team corpus	Current firewall behavior
Unrequested protected file writes	BLOCK or ASK_APPROVAL by authorization.
Credentials in public issue bodies	BLOCK.
Calendar/task creation from read-only requests	BLOCK.
Sensitive file reads the user did not request	BLOCK.
Broadcast email beyond the requested recipient	BLOCK.
Internal *.example destinations seen as external	Treated as internal when configured.
Benign read-only public HTTP GET overblocked	ALLOW.
Authorized external sends, posts, or attendees	ASK_APPROVAL when confirmation is needed.
Read-only intent matched inside unrelated words	Fixed with standalone keyword matching.

6.4 Demonstration Scenarios

Three scenarios exercise the user-visible decision paths and were verified live through the firewall:

Scenario	Tool	Decision	What it demonstrates
Safe email	<code>send_email</code>	ALLOW	Internal email matching the request.
Unauth. HTTP	<code>send_http_request</code>	BLOCK	External state-changing call the user did not authorize.
Delete file	<code>delete_file</code>	ASK_APPROVAL	Legitimate but destructive deletion needing confirmation.

7 Visualizations and Tabular Representations

7.1 Confusion Matrix (94 scenarios)

Rows are the expected label; columns are the firewall’s decision. The perfect diagonal reflects 100% agreement.

Expected \ Firewall	ALLOW	BLOCK	ASK_APPROVAL
ALLOW	30	0	0
BLOCK	0	50	0
ASK_APPROVAL	0	0	14

7.2 Per-Tool Accuracy

Tool	Scenarios	Accuracy
send_http_request	25	100%
send_email	21	100%
delete_file	14	100%
read_file	8	100%
write_file	8	100%
create_github_issue	8	100%
create_calendar_event	6	100%
create_task	4	100%

7.3 Security Console

The React security console provides live visualizations that complement these tables: an attack-simulation view, per-decision cards (ALLOW / BLOCK / ASK_APPROVAL) with audit explanations, metric cards, and a baseline-comparison chart.

8 Links to the Codebase (GitHub)

- **Backend / core:** <https://github.com/CS6180-GenAI-Sec02-Summer2026/agentshield-core>
- **Frontend / console:** <https://github.com/CS6180-GenAI-Sec02-Summer2026/agentshield-console>

9 Milestones and Team Contributions

The project ran over eight weekly milestones. Each milestone contained three parallel stories so all members contributed every week: Mrinal Setty owned the orchestration/backend track, Aditya Shenoy the dataset and red-team track, and Yashas Uttangi the policy, firewall, evaluation, and console track.

Member	Core ownership	GenAI contribution
Mrinal Setty	Orchestration, Target Agent, tool interfaces, backend/API	Multi-agent orchestration, Target Agent prompting, tool-call generation
Aditya Shenoy	Red-Team Agent, dataset, attack taxonomy, dataset quality, report	Adversarial attack generation, injection patterns, synthetic scenarios
Yashas Uttangi	Firewall, policy compiler, evaluation metrics, console	Policy reasoning, audit-explanation generation, LLM-as-a-judge

Work was delivered continuously through reviewed pull requests across all eight milestones. The contribution target was an approximately equal split (about one third each), with each member owning one major story every milestone.

10 Conclusion and Limitations

AgentShield demonstrates that a dedicated, policy-driven firewall secures tool-using AI agents against prompt injection, data exfiltration, and unauthorized actions *before* execution. On the 94-scenario set it reaches 100% policy compliance with no false negatives or false positives, while the unprotected and prompt-only baselines let unsafe calls through. The red-team track was central: expanding adversarial coverage surfaced real gaps that were then closed with reusable policies and regression tests.

Limitations. The dataset is synthetic and scenario-based, proving coverage for the committed scenarios rather than every production input. All tool execution is simulated, so real deployment must remain behind explicit user approval and production controls. `ASK_APPROVAL` is a policy boundary whose calibration depends on the configured policies, and only subjective explanation quality is scored by an LLM-as-a-judge — objective correctness is always measured against fixed ground-truth labels.